

Lecture 20

Trefoil v3: Lists, Cond

Group Work

1. Draw the PST for each of the let expressions
2. Draw the AST for each of the let expressions
3. What should the dynamic environment look like after evaluating this Trefoil program?

```
(define my_pair (cons 1 2))  
(define result1  
  (let  
    ((x (car my_pair))  
     (y (cdr my_pair)))  
    (+ x y)))
```

```
(define a 5)  
(define b 6)  
(define result2  
  (let  
    ((a b)  
     (b a))  
    (cons a b)))
```

Implementing Let

1. AST: | `Let of (string * expr) list`
2. PST->AST: Match on a `Node` whose first child is `Atom "let"` then use a helper function parse all of the bindings to a `(string * expr) list`
3. Interpret: evaluate all of the bindings in the same environment, then evaluate the body in the environment extended with a mapping for each name $\rightarrow$ value pair from the bindings.

Group Work

Sketch out in pseudocode how to implement **cond**:

1. What should the AST constructor look like?
2. How should you parse from PST to AST?
3. How should you interpret?

```
(define (dsum l)
  (cond
    ((nil? l) 0)
    ((cons? l) (+ (dsum (car l))
                   (dsum (cdr l))))
    (true 1)))
```

```
(define (fact n)
  (cond
    ((= n 1) 1)
    (true (* n (fact (- n 1))))))
```

```
(define (length l)
  (cond
    ((nil? l) 0)
    ((cons? l) (+ 1 (length (cdr l))))))
```

Implementing Cond

1. AST: | Cond of (expr * expr) list
2. PST->AST: Match on a Node whose first child is Atom "cond" then use a helper function parse all of the clauses to an (expr * expr) list
3. Interpret: For each clause in order: evaluate the predicate. If it is "truthy" (anything except false), evaluate the right-hand-side of the clause. Otherwise, continue to the next clause. If you run out of clauses, throw an error.